

AI Resume–Job Matching System

GitHub Repository:

<https://github.com/siemngls/ai-resume-job-matcher>

Introduction

The goal of this project is to develop an AI-powered system that evaluates how well a resume matches a job description. Resume screening is a common challenge in recruitment, where companies must efficiently identify candidates whose skills and experience align with job requirements.

This project approaches the problem using Natural Language Processing (NLP), transformer-based semantic similarity models, and AI-generated feedback. Instead of relying only on keyword matching, the system compares the semantic meaning of resumes and job descriptions to better understand candidate relevance.

The application also includes:

- AI-based skill extraction
- Missing skill detection
- Experience matching
- Personalized resume improvement suggestions
- Skill gap roadmap generation
- Automatic job recommendation across multiple jobs

The complete implementation is available in the GitHub repository linked above.

Data

Because large labeled resume–job matching datasets are not publicly available, a custom dataset was created for experimentation and evaluation.

The dataset contains resume and job description pairs labeled as:

- 1 = good match
- 0 = poor match

The dataset includes approximately 30 manually created examples covering multiple domains such as:

- Software Engineering
- Data Science

- Business
- Design
- Marketing

Basic preprocessing steps included:

- Lowercasing text
- Removing extra whitespace
- Cleaning inconsistent formatting

For the automatic job recommendation functionality, additional job description datasets were used and cleaned for semantic retrieval and matching.

One challenge encountered was the relatively small dataset size, which limits generalization and evaluation reliability.

Model & Methods

The system uses transformer-based sentence embeddings from the Sentence Transformers library.

Base Model

all-MiniLM-L6-v2

This model converts resumes and job descriptions into dense vector embeddings that capture semantic meaning.

The system combines several scoring components.

1. Semantic Similarity

Semantic similarity is calculated using cosine similarity between:

- Resume embeddings
- Job description embeddings

This allows the system to understand contextual similarity rather than relying only on exact keyword matching.

2. Skill Matching

The application extracts relevant technical and professional skills from:

- The resume

- The job description

The system then:

- Identifies matched skills
- Detects missing skills
- Computes a skill overlap score

3. Experience Matching

A lightweight experience extraction module detects years of experience mentioned in:

- Resumes
- Job descriptions

The system compares candidate experience with required experience and produces an experience compatibility score.

4. Final Weighted Scoring

The final match score is computed using a weighted combination of:

Component	Weight
Semantic Similarity	50%
Skill Matching	35%
Experience Matching	15%

This hybrid approach provides more balanced scoring than using semantic similarity alone.

5. Fast Semantic Retrieval

To improve performance, precomputed job embeddings are used for fast semantic job search.

Instead of fully analyzing hundreds of jobs one by one:

- The system first retrieves the most semantically similar jobs
- Detailed scoring is then applied only to the top candidates

This significantly improves scalability and responsiveness.

6. AI Feedback & Resume Improvement

The system integrates GPT-based feedback generation using the OpenAI API.

Features include:

- Resume strengths analysis
- Weakness detection
- Personalized improvement suggestions
- Resume rewriting recommendations
- Skill gap learning roadmap generation

This transforms the project from a scoring tool into an interactive career-assistance platform.

Results & Evaluation

The system was evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score

Before Fine-Tuning

Metric	Score
--------	-------

Accuracy	0.83
----------	------

F1 Score	0.80
----------	------

After Fine-Tuning

Metric	Score
--------	-------

Accuracy	0.90
----------	------

F1 Score	0.89
----------	------

The results indicate that fine-tuning improved the model's ability to distinguish between matching and non-matching resume-job pairs.

The system performed best on technical roles where skill overlap was easier to detect, while more general business roles were harder to classify accurately.

However, due to the limited dataset size, these results should be interpreted carefully and cannot yet be considered fully generalizable.

Streamlit Application

A complete interactive web application was built using Streamlit.

The application supports two main modes.

Single Job Analysis

Users can:

- Upload a resume
- Paste a job description
- Receive:
 - Match score
 - Skill analysis
 - Experience comparison
 - AI-generated feedback
 - Resume improvement suggestions
 - Skill roadmap recommendations

Automatic Job Matching

Users can:

- Compare a resume against many jobs
- Automatically retrieve best matching jobs
- View ranked recommendations
- Analyze top matches in detail

The interface also includes:

- Interactive score visualizations
 - Progress indicators
 - Match comparison tables
-

Contributions

All major components of the project were implemented independently, including:

- Dataset creation and labeling
- Semantic matching pipeline
- Skill extraction system
- Experience matching logic
- Fine-tuning setup
- Evaluation pipeline
- Fast semantic retrieval system
- GPT integration
- Resume improvement generator
- Skill roadmap generator
- Streamlit frontend

External Resources Used

The following resources and libraries were used during development:

- Sentence Transformers documentation
- Hugging Face documentation
- Streamlit documentation
- OpenAI API documentation
- Online NLP tutorials

Generative AI tools such as ChatGPT were used for:

- Debugging assistance
- Code structuring guidance
- Architectural suggestions
- UI improvement ideas

Challenges & Future Work

Challenges

The largest challenge was the lack of a large, publicly available labeled dataset for resume–job matching. This required manually creating training and evaluation examples.

Another challenge was balancing:

- Performance
- Accuracy
- Computational efficiency

This led to the use of a lightweight transformer model (MiniLM) instead of larger embedding models.

Handling Streamlit state management and optimizing semantic search performance were also important implementation challenges.

Future Improvements

Potential future extensions include:

- ATS compatibility scoring
- PDF report export
- Cover letter generation
- LinkedIn integration
- Real-time job scraping
- Multi-language support
- Advanced NLP-based skill extraction
- Fine-tuned LLM scoring models
- Resume ranking against multiple candidates
- Personalized learning resource recommendations
- Cloud deployment and database integration